

Ingeniería de Software II



Calidad

Gestión de Calidad

- **Conceptos clave:**
 - La gestión de calidad en software busca **asegurar que los procesos de desarrollo cumplan con estándares definidos** para garantizar la calidad del producto final.
 - Concepto complejo y de aristas múltiples (puntos de vista).
- **Importancia:**
 - **Reduce errores** en etapas tempranas del desarrollo.
 - Mejora la **satisfacción del cliente**.
 - Incrementa la **eficiencia y la sostenibilidad** del software.
- **Métricas de calidad:**
 - Densidad de defectos.
 - Tasa de errores por usuario.
 - Tiempo medio para la resolución de problemas.

LA CALIDAD NO SUCEDE, SE GESTIONA

Calidad - Costo de Calidad y "No Calidad"

- **Costo de calidad (CoQ):** Se refiere a los gastos necesarios para prevenir defectos y garantizar un producto de alta calidad.
 - Ejemplo: Entrenamiento, revisiones, auditorías.
- **Costo de "no calidad":** Gastos derivados de errores o fallos en el producto.
 - Fallos internos: Reparaciones previas al lanzamiento.
 - Fallos externos: Reclamos de clientes, pérdidas de reputación.
- **Conclusión:** Invertir en calidad reduce significativamente los costos de "no calidad".

DIAPOSITIVA MUY IMPORTANTE!!

Caso especial de "No Calidad": Deuda técnica

Definición: la deuda técnica es el costo de retrabajo que aparece cuando se prioriza una solución rápida por sobre la más efectiva.

Origen y analogía: el término lo acuñó Cunningham* (1992) y se compara con la deuda financiera: puede ser “buena” si se gestiona, o “mala” si se ignora.

Cuadrantes de Fowler*: prudente + deliberada, imprudente + deliberada, prudente + inadvertida, imprudente + inadvertida; sirven para calibrar intención y nivel de riesgo.

Origen: puede ser funcional o tecnológica

* Howard Cunningham: XP, Manifiesto Ágil, Wiki

* Martin Fowler: *“Refactoring: Improving the Design of Existing Code”*



DIAPOSITIVA MUY IMPORTANTE!!

Deuda técnica



Cuadrante	¿Qué lo caracteriza?	Ejemplo típico en un proyecto real
Prudente + Deliberada (“ <i>decisión estratégica</i> ”)	El equipo sabe que genera deuda y planea cómo y cuándo pagarla .	Sprint de lanzamiento: para llegar a la demo con el cliente, se omite la capa de validaciones complejas y se documenta una “tarjeta” de backlog con el refactor previsto para el próximo ciclo.
Imprudente + Deliberada (“ <i>lo hacemos rápido y después vemos</i> ”)	Se conoce el riesgo pero se acepta sin plan de pago; prima la presión de negocio.	Parche de producción a las 3 a. m.: se mete código hardcodeado con credenciales para reactivar un servicio crítico; “mañana lo limpiamos”... y pasa un año .
Prudente + Inadvertida (“ <i>nadie podía preverlo</i> ”)	Se trabajó con buenas prácticas, pero apareció información nueva .	Cambio de normativa: se construye un módulo fiscal según la ley vigente; tres meses después sale una nueva resolución impositiva que obliga a reescribir gran parte de la lógica.
Imprudente + Inadvertida (“ <i>no sabíamos que era tan grave</i> ”)	Falta de conocimiento o control; la deuda surge sin que el equipo se dé cuenta.	On-boarding apurado: desarrolladores juniors copian-pegan código de internet sin tests ni revisión; meses más tarde aparecen bugs y duplicación masiva de lógica en toda la base de código.

Caso especial de "No Calidad": Deuda técnica

Tipos según Steve McConnell (autor de "Code complete"):

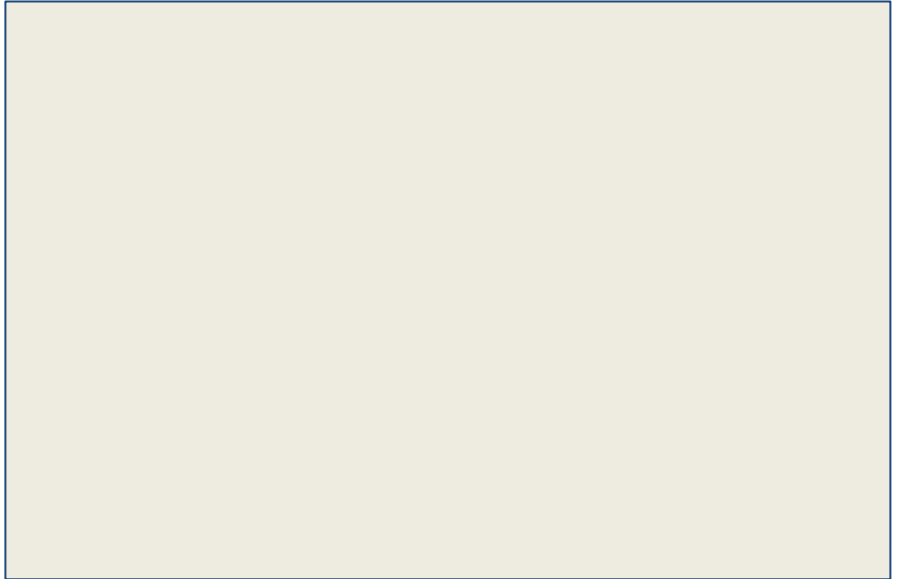
- **Intencionada:** decisión consciente de optimizar el presente (puede ser a corto o largo plazo).
 - Ejemplo: elegir un **framework rápido pero de bajo rendimiento** para llegar a un deadline; requerirá reescritura posterior.
- **No intencionada:** errores, falta de conocimiento o diseño defectuoso.
 - Ejemplo: liberar código escrito por **desarrolladores junior sin revisión suficiente**, generando errores que aparecen tras el lanzamiento.

Buenas prácticas de control:

- **Registrar** cada deuda en un sistema de seguimiento/backlog con esfuerzo y fecha estimada.
- Tratarla como historias de usuario (en enfoques ágiles); si supera 90 días, **marcarla como crítica**.
- Fomentar transparencia y comunicación: **pagar la deuda en incrementos pequeños y visibles** para todo el equipo.

Calidad - Proceso de Gestión de Calidad

- **Fases principales:**
 - Planificación de calidad: incluir en el plan
 - Control de calidad (QC)
 - Aseguramiento de calidad (QA)
- **Herramientas utilizadas:**
 - Diagramas de causa-efecto (Diagrama de Ishikawa o “fishbone”)
 - Auditorías internas.
 - Indicadores clave de rendimiento (KPIs).
- **Ciclo de mejora continua:** Planificar, Hacer, Verificar, Actuar (PDCA).



DIAPOSITIVA MUY IMPORTANTE!!

Calidad - Control de Calidad (QC)

Enfoque

- **Producto** y entregables concretos
- **Detección** de defectos existentes

Actividades

- **Revisiones** de código
- **Pruebas** (unitarias, integración, sistema)
- **Verificación** de requisitos

Resultado

- **Lista de defectos** e incidencias
- **Métricas** (defect density, severidad)

DIAPOSITIVA MUY IMPORTANTE!!

Calidad - Aseguramiento de Calidad (QA)

Enfoque

- **Procesos** y metodologías de desarrollo
- **Prevención** de defectos

Actividades

- Definición de **políticas y estándares** (ISO/IEC 25010, CMMI)
- **Auditorías** de proceso
- Capacitación y mejora continua

Resultado

- **Procesos** documentados y validados
- Confianza en la calidad durante **todo el ciclo de vida**

Calidad - Control vs. Aseguramiento

Perspectiva

- QC: Orientado al producto
- QA: Orientado al proceso

Objetivo

- QC: Detectar defectos
- QA: Prevenir defectos

Momento

- QC: Durante y después del desarrollo
- QA: Desde el inicio y durante todo el ciclo

Responsabilidad

- QC: Equipo de testing/inspección
- QA: Toda la organización (equipo SQA -software quality assurance- facilita)

Calidad - Verificación y Validación

- **Verificación:** Asegura que el **producto** se esté **construyendo correctamente** según los requisitos especificados.
 - Ejemplo: Revisiones de código, inspecciones.
- **Validación:** Comprueba que el **producto** final **cumple** con las **necesidades** del cliente.
 - Ejemplo: Pruebas funcionales, pruebas de usuario.
- **Técnicas comunes:**
 - Revisiones técnicas.
 - Pruebas de integración.
 - Simulaciones.

Pruebas de software - Clasificación de las Pruebas 1/2

Según el *nivel*:

- **Pruebas unitarias:** Validan componentes individuales.
- **Pruebas de integración:** Aseguran que los módulos funcionan juntos.
- **Pruebas del sistema:** Verifican el software completo.
- **“Smoke test” (*)**: Conjunto reducido de pruebas de sistema
- **Pruebas de aceptación:** Realizadas por el cliente para validar requisitos.

() Se la llama así porque es “ver si sale humo” apenas se enciende el sistema: si falla, no vale la pena seguir probando.*

Según la *técnica*:

- **Pruebas estáticas:** Evaluación sin ejecutar el código (revisiones, inspecciones).
- **Pruebas dinámicas:** Ejecución activa para detectar defectos.

Pruebas de software - Clasificación de las Pruebas 2/2

Según el *alcance*:

- **Funcionales:** Validan **funcionalidades** específicas.
- **No funcionales:** Evalúan aspectos como **rendimiento**, **seguridad**, **usabilidad**, etc.

Según la *automatización*:

- **Manuales:** Realizadas por **testers**.
- **Automatizadas:** Usan **scripts** y **herramientas**.

Pruebas de sistema / No funcionales:

- **Pruebas de recuperación:** recuperarse de fallas y **reanudar** el procesamiento
- **Pruebas de seguridad:** intenta verificar que los mecanismos de **protección** (*pentest*)
- **Pruebas de compatibilidad/despliegue:** varias plataformas y bajo **más de un entorno** de sistema operativo.
- **Pruebas de esfuerzo/rendimiento:** demanda **recursos** en cantidad, frecuencia o volumen anormales
- **Pruebas de usabilidad:** analizan la **experiencia** del usuario.
- **Pruebas de escalabilidad:** comprueban cómo el sistema responde al **crecimiento**.
 - Escalabilidad **vertical:** aumentar los **recursos** de un solo servidor (ej: más CPU, memoria)
 - Escalabilidad **horizontal:** agregar más **servidores** para compartir la carga

Pruebas de software - Técnicas y Estrategias de Pruebas

- **Caja negra:** Basada en especificaciones sin conocer el código.
- **Caja blanca:** Basada en la estructura del código.
- **Técnicas híbridas:** Combinan caja negra y caja blanca.
- **Generación de casos de prueba efectivos:**
 - Identificar escenarios clave.
 - Usar técnicas como partición de equivalencia y análisis de valores límite.
- **Gestión de riesgos:**
 - Priorización según impacto y probabilidad de fallos.

Documentos “““importantes””” (1/3)

Plan de Gestión de Calidad de Software: cómo se asegura y controla la calidad durante todo el ciclo de vida del software. Incluye estrategias, métricas, responsabilidades, y actividades para garantizar que el producto cumpla con los estándares establecidos.



Documentos “““importantes””” (2/3)

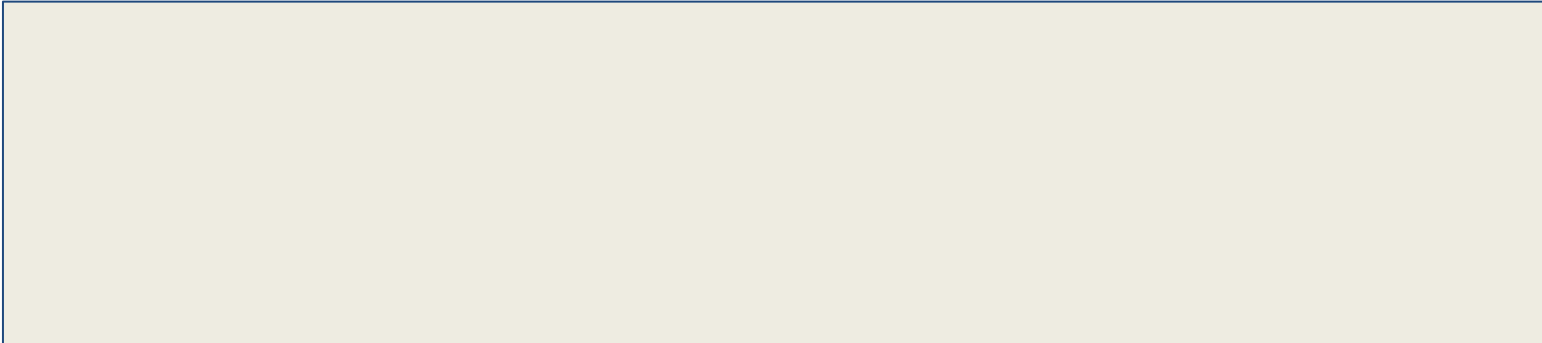
Plan de Pruebas de Software: enfoque, alcance, recursos y cronograma para realizar las pruebas. Describe los tipos de pruebas, los criterios de aceptación y finalización, el ambiente de pruebas y las herramientas a utilizar.

Documentos “““importantes””” (3/3)

Casos de Prueba de Software: pasos específicos para probar una funcionalidad o característica del software. Incluye precondiciones, datos de entrada, resultados esperados y obtenidos, para garantizar que cada componente funcione como se espera. Incluye el registro y trazabilidad de las pruebas.

Objetivo:

- Verificar que **cada requisito de negocio se cumpla** sin errores.
- **Detectar defectos antes de la puesta en producción**, reduciendo costo de corrección.
- Servir como **documentación viva** que facilita mantenimiento y regresión.



Casos de Prueba - Formato Given-When-Then

¿Qué es?

- Plantilla de Behavior-Driven Development que **conecta requisitos de negocio con pruebas ejecutables**.
- Favorece **lenguaje comprensible** para negocio, QA y desarrollo.

Estructura

*Given (contexto): **estado previo del sistema y datos iniciales**. When (acción): **evento/disparo que el usuario o sistema realiza**. Then (resultado): **salida observable o cambio esperado**.*

Buenas prácticas

- **Una sola acción principal** por escenario.
- **Evitar condicionales:** usar escenarios separados para ramas positivas y negativas.
- Mantener las **frases en tiempo presente y voz activa** (el sujeto realiza la acción).